



Facultad de Ingeniería
Departamento de Ingeniería en Computación e
Informática

Análisis e interpretación: Algoritmos recursivos de búsqueda y ordenamiento

Ayrton Morrison

Iván Gallardo

Pablo Gómez

Ingeniería Civil en Computación e Informática

Docente: MSc. Jackeline Aldridge

Ramo: Diseño de Algoritmos

RESUMEN

Este informe presenta el desarrollo de un sistema en lenguaje C para la gestión de información de deportistas, capaz de generar, almacenar, ordenar y buscar registros en archivos CSV. Para ello, se implementaron los algoritmos de ordenamiento Bubble Sort optimizado, Insertion Sort, Selection Sort optimizado y Cocktail Shaker Sort, junto con los algoritmos de búsqueda secuencial y búsqueda binaria iterativa. Además, se realizaron experimentos para analizar el comportamiento teórico y empírico de estos algoritmos en distintos tamaños de entrada y en diferentes casos de ejecución. Los resultados obtenidos permitieron comparar su rendimiento mediante benchmarks y gráficos, facilitando la comprensión de sus diferencias en eficiencia y complejidad.

ÍNDICE GENERAL

ÍNDICE DE FIGURAS

INTRODUCCIÓN

"**Divide y Vencerás**" es un paradigma de programación, que consiste en un conjunto de técnicas algorítmicas en las que primero se divide el problema a resolver en subproblemas de menor tamaño y de la misma naturaleza, luego se resuelven los subproblemas de manera independiente y recursiva, hasta llegar a un caso base (que es un caso tan trivial que detiene la recursión), y finalmente se combinan los resultados para llegar a la solución final.

En el presente informe, se analizará empíricamente la complejidad algorítmica de varios algoritmos que utilizan este paradigma de programación, y evaluar el impacto que puede tener cada variante de cada algoritmo.

Los algoritmos que serán objeto de este análisis involucrarán tanto algoritmos de ordenamiento, como algoritmos de búsqueda, como también un algoritmo de selección.

Estos son:

ORDENAMIENTO

- *Merge Sort*.
- *Merge-Insertion Sort* (versión de *Merge Sort* que incluye *Insertion Sort* cuando se llegan a tener subarreglos de tamaño pequeño).
- *Quick Sort* con pivote en la primera posición.
- *Quick Sort* con pivote en la última posición.
- *Quick Sort* con pivote en una posición aleatoria.
- *Quick Sort* escogiendo al pivote como resultado de una mediana entre 3 elementos.

BÚSQUEDA

- Búsqueda binaria clásica.
- Búsqueda binaria para rangos.
- Búsqueda exponencial.
- Búsqueda por interpolación.

SELECCIÓN

- *Quick Select*.

OBJETIVOS

2.1. Objetivo Principal

2.2. Objetivos Secundarios

MARCO TEÓRICO

DESARROLLO

4.1. Diseño general del sistema

El sistema fue diseñado para gestionar información de deportistas de forma simple y modular, permitiendo generar datos, cargarlos desde archivos CSV y aplicar sobre ellos distintas operaciones de ordenamiento, búsqueda y análisis experimental. Para facilitar su desarrollo y comprensión, la implementación se organizó en componentes que separan la representación de los datos, la generación y carga de registros, y las funcionalidades principales ofrecidas por el programa.

4.1.1. Estructura de datos de los deportistas

La estructura de datos utilizada para representar a cada deportista se definió como una estructura con campos específicos para almacenar distinta información; estos campos corresponden a:

- **ID:** Identificador único del deportista.
- **Nombre:** Nombre completo del deportista.
- **Equipo:** Equipo al que pertenece.
- **Puntaje:** Puntuación total acumulada.
- **Competencias:** Número de competencias en las que ha participado.

Cada parámetro permite almacenar información, la cual puede ser utilizada para ordenar o buscar a los deportistas según distintos criterios. Esta estructura se implementó utilizando un `struct` en C, lo que facilita su manejo y acceso a los campos correspondientes durante las operaciones de ordenamiento y búsqueda.

4.1.2. Generación y carga de datos

La generación de datos de los parámetros asociados a deportistas se realizó mediante distintas funciones, las cuales permiten crear u obtener estos registros de manera pseudo-aleatoria. La generación de un nombre se realizó permitiendo seleccionar aleatoriamente letras de un conjunto predefinido, escogiendo un tamaño de 3 a 8 caracteres. Para el equipo,

se seleccionó aleatoriamente entre un conjunto específico de equipos de Esports. El ID se generó en forma incremental, asegurando la unicidad de cada registro. Por otro lado, el puntaje y la cantidad de competencias se generaron utilizando funciones de generación de números aleatorios dentro de rangos predefinidos. Esta metodología permitió crear un conjunto de datos variado y representativo para probar las funcionalidades del sistema. Posterior a la creación de los registros, estos se desordenaron utilizando el algoritmo de Fisher-Yates para garantizar que los datos no se encuentren en un orden específico, lo que es fundamental para evaluar correctamente el desempeño de los algoritmos de ordenamiento y búsqueda implementados. Finalmente, los registros generados se almacenaron en un archivo CSV, lo que facilita su posterior carga y manipulación dentro del programa.

CONCLUSIÓN

ANEXOS

6.1. Código fuente

6.2. Contribución por integrante